

Skill ossp

Name: Lokesh Tella

Roll no : 2520090105

```
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];

    while (1)
    {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting...\n");
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

Skill ossp

Task 2:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd1, fd2;
    char buffer[100];
    int n;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 < 0)
    {
        perror("Error opening File1.txt");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 < 0)
    {
        perror("Error creating File2.txt");
        close(fd1);
        return 1;
    }

    n = read(fd1, buffer, sizeof(buffer));

    if (n < 0)
    {
        perror("Error reading File1.txt");
        close(fd1);
        close(fd2);
        return 1;
    }

    write(fd2, buffer, n);

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");

    return 0;
}
```

Skill ossp

Skill ossp

Task 3:

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <termios.h>

int main()
{
    struct termios oldt, newt;
    char ch;
    char buffer[100];
    int index = 0;

    tcgetattr(STDIN_FILENO, &oldt);

    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("Enter command: ");

    while (1)
    {
        read(STDIN_FILENO, &ch, 1);

        if (ch == '\n' || ch == '\r')
        {
            buffer[index] = '\0';
            printf("\nYou entered: %s\n", buffer);
            break;
        }

        if (ch == 127)
        {
            if (index > 0)
            {
                index--;
                printf("\b \b");
                fflush(stdout);
            }
        }
        else
        {
            if (index < 99)
            {
                buffer[index++] = ch;
                printf("%c", ch);
                fflush(stdout);
            }
        }
    }

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return 0;
}
```

Skill ossp

Task 4:

Skill ossp

```
GNU nano 7.2 task4.c
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

int main()
{
    char history[MAX_HISTORY][MAX_LENGTH];
    char input[MAX_LENGTH];

    int history_count = 0;
    int i;

    while (1)
    {
        printf("myshell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strlen(input) > 0)
        {
            if (history_count < MAX_HISTORY)
            {
                strcpy(history[history_count], input);
                history_count++;
            }
            else
            {
                for (i = 0; i < MAX_HISTORY - 1; i++)
                {
                    strcpy(history[i], history[i + 1]);
                }

                strcpy(history[MAX_HISTORY - 1], input);
            }
        }

        printf("Command: %s\n", input);

        printf("\nCommand History:\n");

        for (i = 0; i < history_count; i++)
        {
            printf("%d: %s\n", i + 1, history[i]);
        }

        return 0;
    }
}
```

Skill ossp

Task 5:

```
GNU nano 7.2 task5.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node
{
    char data[100];
    struct Node *next;
};

int main()
{
    struct Node *head = NULL;
    struct Node *newNode;
    struct Node *temp;

    int n, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    getchar();

    for (i = 0; i < n; i++)
    {
        newNode = (struct Node *)malloc(sizeof(struct Node));

        if (newNode == NULL)
        {
            printf("Memory allocation failed.\n");
            return 1;
        }

        printf("Enter data for node %d: ", i + 1);
        fgets(newNode->data, sizeof(newNode->data), stdin);
        newNode->data[strcspn(newNode->data, "\n")] = '\0';

        newNode->next = head;
        head = newNode;
    }

    printf("\nLinked List:\n");
    temp = head;

    while (temp != NULL)
    {
        printf("%s\n", temp->data);
        temp = temp->next;
    }

    temp = head;

    while (temp != NULL)
    {
        struct Node *next = temp->next;
        free(temp);
        temp = next;
    }

    printf("\nMemory released successfully.\n");
    return 0;
}
```

Skill ossp